WatchesDen

Architecture, API Flow & Database Walkthrough

Demo preparation guide with likely panel questions and answers

PDI Talent Pool - Competency Assessment - Use Case 2: E-Commerce Order & Inventory

What this document is for

A single reference to walk the panel through the system and to answer technical questions. It covers: the architecture and request flow, every API flow step by step, the database structure, every library and tool used (with versions and the reason for each), a timed demo script, and the questions a panel is most likely to ask, with short answers that point to the exact code.

Item	Value
Web app	http://localhost:3000
Swagger UI (all services)	http://localhost:8080/swagger-ui.html
API gateway	http://localhost:8080
Eureka dashboard	http://localhost:8761
Admin login	admin@watchesden.com / Admin@123
Start everything	docker compose up --build
Run all tests	cd backend && mvn verify ; cd frontend && npm test ; npx newman run postman/...
Test status	45 unit + 21 integration (backend), 18 frontend, 43 Postman requests / 116 assertions: all passing

Contents

1. Project at a glance 4

2. Technologies and libraries 5

2.1 Backend . . . 5

2.2 Testing . . . 6

2.3 Frontend . . . 6

2.4 DevOps and infrastructure . . . 6

3. Architecture walkthrough 7

3.1 Components . . . 7

3.2 Why the services are split this way . . . 7

3.3 MVC inside every service . . . 8

4. Technical flow walkthrough (one request end to end) 9

Step by step . . . 9

4.1 Security model . . . 9

5. API flow walkthroughs 10

5.1 Register and login . . . 10

5.2 Browse, search and product details . . . 10

5.3 Customize and add to cart . . . 11

5.4 Checkout (the saga) . . . 11

5.5 Order history and cancellation . . . 13

5.6 Admin flows . . . 13

5.7 Complete API reference . . . 13

6. Database structure 14

6.1 Tables in detail . . . 14

6.2 Migrations (Flyway) . . . 15

7. Concurrency, consistency and events 17

7.1 Events and messaging . . . 17

7.2 Scalability . . . 17

7.3 Error handling, validation and logging . . . 17

8. Testing, Docker and CI/CD 19

8.1 Test pyramid . . . 19

8.2 Docker . . . 19

8.3 CI/CD pipeline (GitHub Actions) . . . 19

9. Demo script (about 12 minutes) 20

10. Likely panel questions and answers 21

Architecture . . . 21

Security . . . 21

Data and transactions . . . 21

Concurrency 22
Messaging, scaling and resilience 22
Quality and delivery 23
11. Assumptions, limitations and next steps **24**
Roadmap summary 24
Where to find things in the repository 24

1. Project at a glance

WatchesDen is an online watch store where customers can buy watches as they are or **customize** them (case, dial, strap, hands). It implements the assessment's **Use Case 2: E-Commerce Order & Inventory**, whose focus is **events, messaging, concurrency and scalability**.

Requirements from the brief and where each one is met:

Requirement	How it is met
MVC architecture	Every service: controller -> service -> repository -> model, DTOs at the boundary
REST APIs	20+ endpoints behind one gateway, documented with OpenAPI / Swagger
Clean, structured project	Maven multi-module backend, shared common library, React app split into pages / components / context / api
Error handling, validation, logging	Global exception handler with one error JSON shape; Bean Validation on all request DTOs; correlation-ID logging across services
Unit tests passing	45 JUnit 5 + Mockito tests (backend), 18 Vitest tests (frontend)
Functional / integration tests passing	21 Spring Boot integration tests + Postman/Newman suite of 43 requests / 116 assertions
CI/CD pipeline	GitHub Actions: build, test, Docker build, full-stack Newman run, push images to GHCR
E-commerce minimum features	Register/login, listing & search, details, cart, checkout, order history, admin product CRUD
Extras	Watch customization, order cancellation with stock return, admin order fulfilment, idempotent checkout

One-line pitch for the panel

Say: *"It is a microservice e-commerce system: an API gateway in front of three Spring Boot services, each with its own PostgreSQL database. The interesting part is checkout: stock is reserved atomically so we can never oversell, the order is saved with a compensating action if anything fails, and retries are safe because of an idempotency key."*

2. Technologies and libraries

Versions below are the exact ones packaged in the build (read from the built JAR files and node_modules).

2.1 Backend

Library / tool	Version	Used for	Why we chose it
Java (target)	17 LTS	All services	Long-term support; records and text blocks keep DTOs and SQL short
Spring Boot	3.3.4	Base of every service	Auto-configuration, embedded server, production features (actuator)
Spring Framework (core / web MVC)	6.1.13	Controllers, DI, transactions	Industry standard; MVC model required by brief
Embedded Tomcat	10.1.30	HTTP server in each service	Comes with spring-boot-starter-web
Spring Cloud	2023.0.3	Microservice infrastructure (BOM)	Version set that matches Boot 3.3
Spring Cloud Gateway	4.1.5	api-gateway: routing, filters, CORS	Reactive (Netty), non-blocking, simple route config
Netty / Reactor Netty	4.1.113 / 1.1.22	Gateway runtime	Non-blocking I/O for the single entry point
Netflix Eureka (server + client)	4.1.3 (client 2.0.3)	Service registry and discovery	Services find each other by name; no hard-coded hosts
Spring Cloud LoadBalancer	4.1.4	lb:// routes, @LoadBalanced RestClient	Client-side load balancing across instances
Spring Data JPA	3.3.4	Repositories, Specifications, paging	Less boilerplate; type-safe dynamic search queries
Hibernate ORM	6.5.3.Final	JPA implementation	Default in Boot; supports @Version optimistic locking
HikariCP	5.1.0	DB connection pool	Fast default pool in Spring Boot
PostgreSQL JDBC driver	42.7.4	Database access	Production database
Flyway (+ PostgreSQL module)	10.10.0	Versioned schema migrations	Repeatable, reviewable schema; app refuses to start if schema drifts
Hibernate Validator (Bean Validation)	8.0.1 (API 3.0)	@NotBlank, @Email, @Pattern ... on DTOs	Declarative input validation with field-level errors
jjwt	0.12.6	Issue and verify HS256 JWT	Small, well-maintained JWT library
spring-security-crypto	6.3.3	BCrypt password hashing	Only the crypto module, not full Spring Security (auth is done at gateway)
springdoc-openapi (webmvc + webflux)	2.6.0	OpenAPI spec and Swagger UI	Generates docs from code; gateway aggregates all services
Swagger UI	5.17.14	Interactive API docs	Bundled by springdoc
Jackson	2.17.2	JSON (de)serialization	Default in Spring
SLF4J + Logback	Logback 1.5.8	Logging with MDC correlation IDs	Default in Spring Boot
Spring Boot Actuator / Micrometer	- / 1.13.4	/actuator/health for Docker and CI	Health checks, metrics ready

2.2 Testing

Library / tool	Used for
JUnit 5 (Jupiter)	Test framework for all backend tests
Mockito	Mocks for repositories and the product client in unit tests
AssertJ	Readable assertions
Spring Boot Test + MockMvc	Integration tests: full context, real HTTP layer, real Flyway migrations
H2 database (PostgreSQL mode)	Fast in-memory DB for integration tests
Maven Surefire / Failsafe	Runs *Test (unit) in test phase and *IT (integration) in integration-test phase
JaCoCo 0.8.12	Code coverage report
Vitest 2.1.9 + Testing Library 16 + jsdom 25	Frontend unit and component tests
Postman + Newman 6	Black-box functional tests through the gateway on the running stack

2.3 Frontend

Library / tool	Version	Used for
React + React DOM	18.3.1	UI components and hooks
React Router	6.30	Pages and protected routes
Vite (+ @vitejs/plugin-react)	5.4 (4.7)	Dev server with /api proxy, production build
fetch API (no axios)	built-in	HTTP calls through a small wrapper (api/client.js)
React Context	built-in	Auth, Cart and Toast state (no Redux needed)
Plain CSS	-	Simple, intentionally basic styling; focus is the backend

2.4 DevOps and infrastructure

Tool	Used for
PostgreSQL 16 (alpine image)	Database, one instance with three databases
Docker (multi-stage builds)	One Dockerfile for all Java modules (ARG SERVICE); node build + nginx for the frontend
eclipse-temurin:17-jre-alpine	Small Java runtime image, runs as non-root user
nginx 1.27	Serves the React build and proxies /api to the gateway
Docker Compose	Starts the whole stack in dependency order with health checks
GitHub Actions	CI/CD pipeline
GitHub Container Registry (GHCR)	Stores built images (main branch)

3. Architecture walkthrough

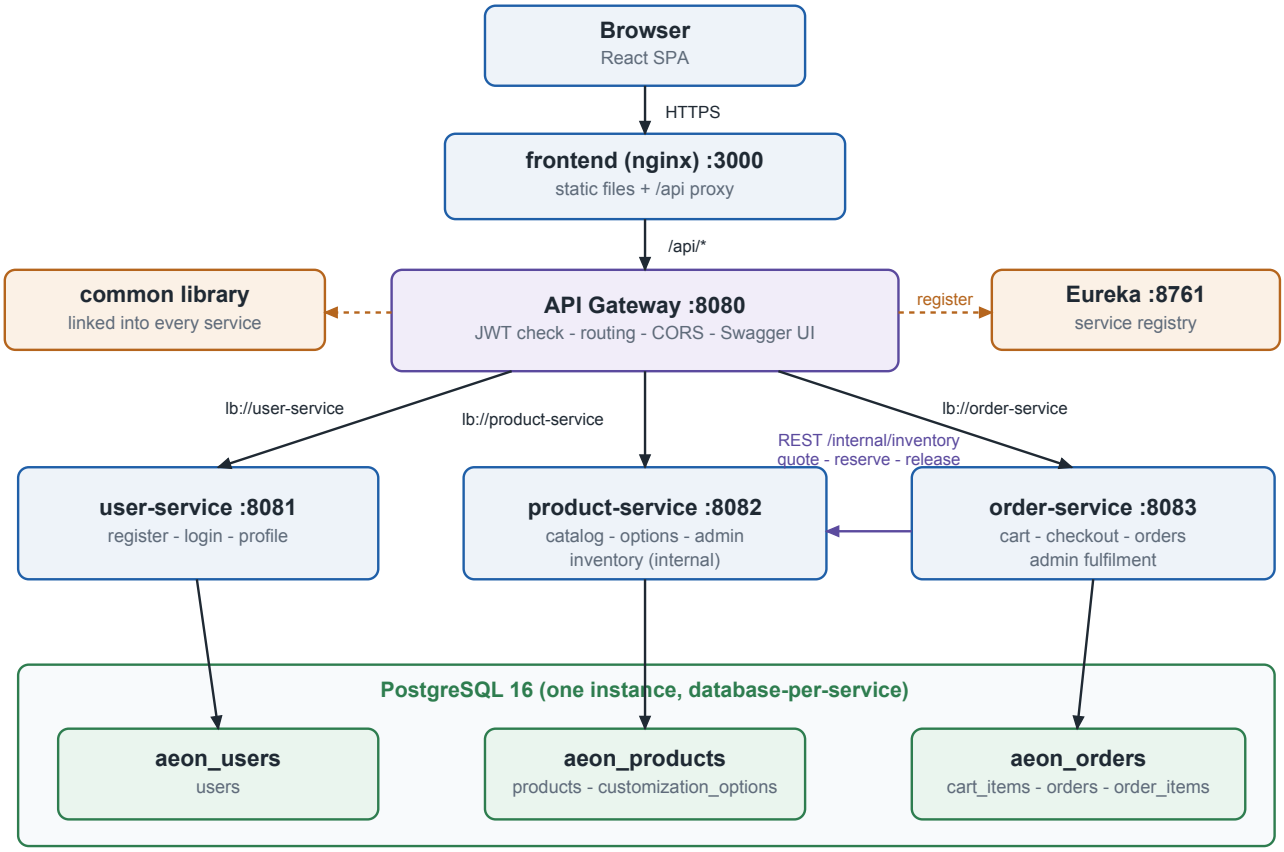


Figure 1. Components, ports and data ownership. Solid arrows are HTTP calls; dashed orange arrows are Eureka registration.

3.1 Components

Component	Port	Responsibility
frontend (nginx)	3000	Serves the React app. Proxies /api/* to the gateway, so the browser talks to one origin.
api-gateway	8080	Single entry point. Validates JWT, removes spoofed identity headers and adds trusted ones, routes to services via Eureka, CORS, correlation ID, aggregated Swagger UI.
discovery-server (Eureka)	8761	Registry. Every service registers itself; the gateway and order-service look services up by name.
user-service	8081	Registration, login (issues JWT), profile, BCrypt hashing, seeds the admin account.
product-service	8082	Catalog, search, customization options, admin product CRUD, and the internal inventory API (quote / reserve / release stock).
order-service	8083	Cart, checkout saga, order history, cancellation, admin status changes, order events.
common (library, not a running service)	-	Shared code: error model + global exception handler, JWT utility, identity-header contract, correlation-ID filter, pagination envelope. Wired in by Spring Boot auto-configuration.
PostgreSQL	5432	Three databases: aeon_users, aeon_products, aeon_orders. Each service owns one.

3.2 Why the services are split this way

- **One service per business capability and per data owner**: identity, catalog + stock, purchase (cart + orders).
- **Stock lives with the product**, because every stock rule (reserve, release, never negative) needs the product row. Putting stock in a separate service would need distributed locking.
- **Cart lives with orders**, because checkout turns the cart into an order in one local transaction.
- **Gateway + Eureka** mean clients only know one URL and services can scale or move without config changes.

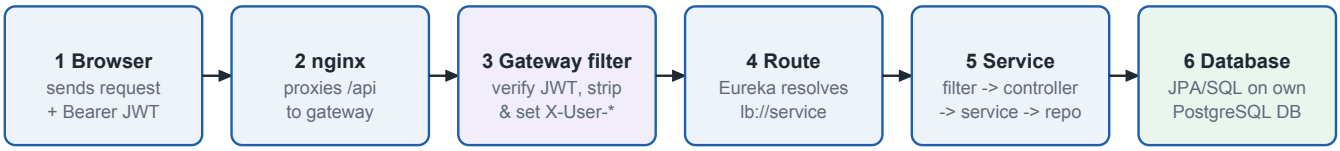
3.3 MVC inside every service

```
com.aeon.<service>
  controller   HTTP: routes, @Valid input, reads identity headers, calls the service
  service      business rules and @Transactional boundaries
  repository   Spring Data JPA interfaces, Specifications, atomic UPDATE queries
  model        JPA entities (the "M")
  dto          request/response records (the JSON "view" of the model)
  client       (order-service) typed HTTP client for product-service
  event        (order-service) domain event + after-commit listener
  config       beans, OpenAPI info
```

Rule we follow: controllers never call repositories, and entities never leave the service layer; they are converted to DTO records first. This keeps the API stable when the database changes.

> **Say:** *"Each service is a small MVC application. The model is the JPA entity, the controller handles HTTP, and the view is the JSON DTO. Business logic sits in the service layer, which also owns the transaction."*

4. Technical flow walkthrough (one request end to end)



Response travels back the same way with the same X-Correlation-Id header.
Errors at any step use one JSON shape: {timestamp, status, error, message, path, correlationId, fieldErrors}.
Gateway rejects early: no/invalid token on a protected path = 401, non-admin on /api/admin/** = 403.

Figure 2. Lifecycle of an authenticated request such as GET /api/cart.

Step by step

#	Where	What happens	Code
1	Browser	React calls fetch("/api/cart") with header Authorization: Bearer <JWT> (token saved after login).	frontend/src/api/client.js
2	nginx / Vite	Proxies /api/* to the gateway on :8080 (same origin, so no CORS issue in production).	frontend/nginx.conf, vite.config.js
3	Gateway	JwtAuthenticationFilter (GlobalFilter, order -100): verifies signature and expiry, applies RoutePolicy (public / user / admin), deletes any client-sent X-User-* headers, adds X-User-Id, X-User-Email, X-User-Role and X-Correlation-Id.	api-gateway/.../JwtAuthenticationFilter.java, RoutePolicy.java
4	Gateway	Route lb://order-service matches /api/cart/**; Spring Cloud LoadBalancer asks Eureka for a live instance.	api-gateway/.../application.yml
5	order-service	CorrelationIdFilter puts the ID into the logging MDC. CartController reads X-User-Id, CartService loads cart lines and asks product-service for live prices.	common/.../CorrelationIdFilter.java, CartController, CartService
6	Database	Spring Data JPA queries aeon_orders.cart_items through HikariCP.	CartItemRepository
7	Back	DTO record serialized to JSON by Jackson; response carries the same X-Correlation-Id. Any exception is converted by GlobalExceptionHandler.	common/.../GlobalExceptionHandler.java

4.1 Security model

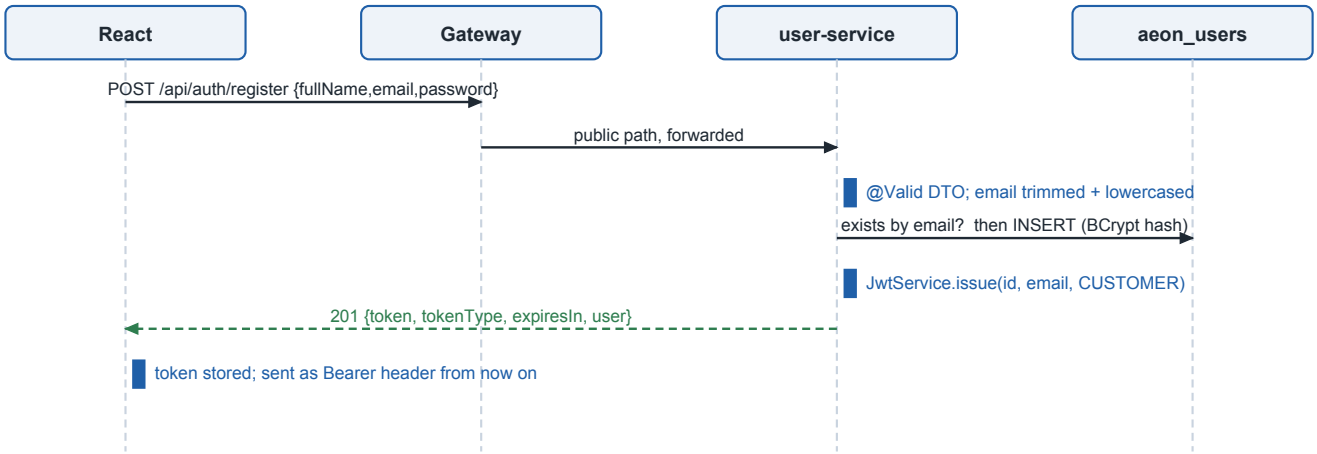
Path	Rule at the gateway
/api/auth/**, Swagger, /actuator/health, all OPTIONS	Public
GET /api/products/**, GET /api/customizations	Public (browsing does not need login)
/api/admin/**	Valid token AND role ADMIN, otherwise 403
All other routed paths (cart, orders, profile)	Valid token, otherwise 401
/internal/** (service-to-service)	No route exists, so the gateway returns 404; only reachable inside the Docker network

- **JWT**: HS256, signed with JWT_SECRET (minimum 32 bytes, checked at startup). Claims: sub = user id, email, role. Valid 24 hours.
- **Passwords**: BCrypt hash; plain text is never stored or logged. Same 401 message for wrong email or wrong password (no account enumeration).
- **Defence in depth**: services check again (AuthHeaders.requireUser / requireAdmin). Another user's order returns 404, not 403, so IDs cannot be probed.
- **Header spoofing** is blocked: a customer token sent with X-User-Role: ADMIN still gets 403 (covered by a Postman test).

5. API flow walkthroughs

Every flow below goes through the gateway on :8080. "Gateway" in the diagrams means the JWT filter plus routing. Solid arrows are requests; dashed green arrows are responses.

5.1 Register and login



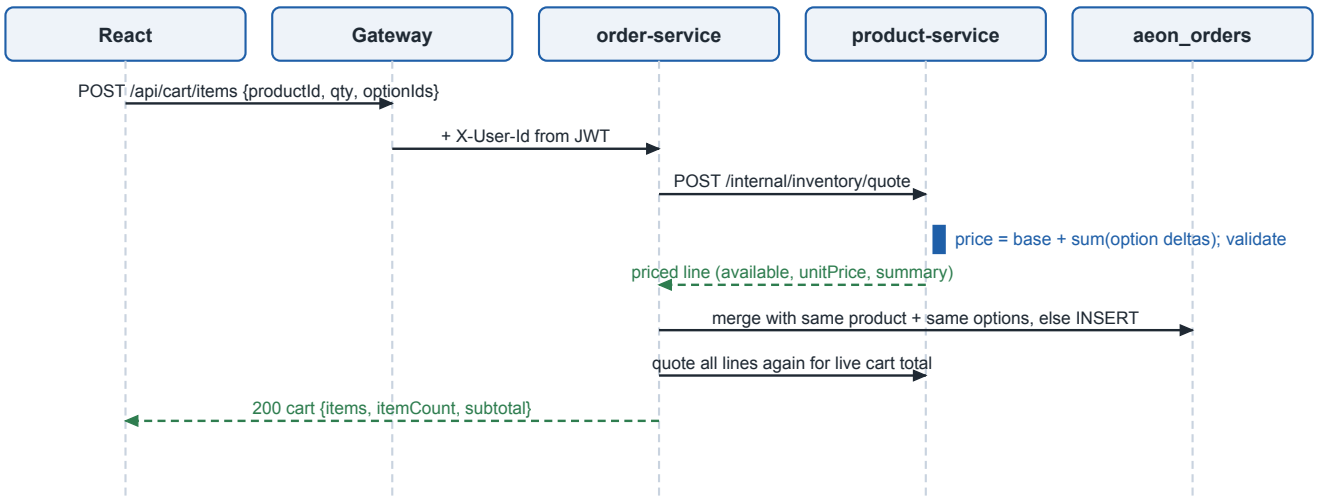
Case	Result
Email already registered	409 "An account with this email already exists"
Invalid input (bad email, short password)	400 with fieldErrors {email, password, fullName}
Login with wrong password or unknown email	401 "Invalid email or password" (same message both cases)
GET /api/users/me	Profile of the user in the token

5.2 Browse, search and product details

GET /api/products?q=&collection=&minPrice=&maxPrice=&customizable=&page=&size=&sort= is public. ProductService combines small JPA Specifications (activeOnly AND matchesText AND inCollection AND priceBetween AND customizable). A null filter is skipped. Text search covers name, collection, description, case material and movement. Results are wrapped in PageResponse {content, page, size, totalElements, totalPages}. Page size is capped at 50.

GET /api/products/{id} returns only active products (404 otherwise). **GET /api/products/collections** returns distinct names for the filter dropdown. **GET /api/customizations** returns options grouped by part.

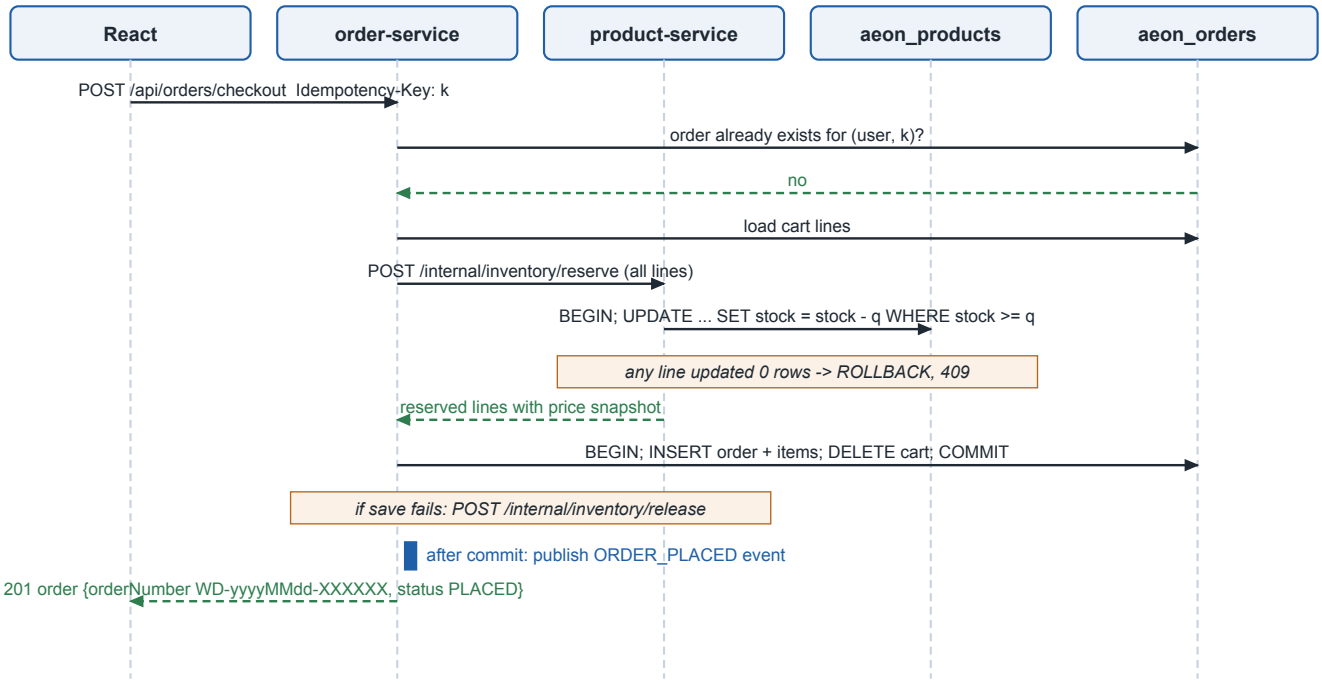
5.3 Customize and add to cart



Why it is designed this way:

- **The browser never sends prices**; only option IDs. The server computes the price, so it cannot be tampered with.
- product-service rejects: options for a non-customizable product, unknown option IDs, and two options for the same part (400).
- Option IDs are stored **sorted**, so "case-ti, dial-aurora" and "dial-aurora, case-ti" merge into one cart line.
- Limits: max 10 units per line, max 20 lines. If a product is later removed, the cart shows it as "not available" instead of failing.

5.4 Checkout (the saga)



Step	Detail
1. Idempotency check	If the same user sends the same Idempotency-Key again (double click, retry), the original order is returned and nothing is reserved again.
2. Reserve stock	One transaction in product-service. Lines are processed in product-ID order (avoids deadlocks). Each decrement is a single conditional UPDATE, so stock can never go below zero. If any line is short, everything rolls back and the user sees "Insufficient stock for <name>".

Step	Detail
3. Save order	Local transaction in order-service: order + items (name and price copied as a snapshot) are inserted and the cart is cleared.
4. Compensate	If step 3 fails, stock reserved in step 2 is released. A unique-constraint clash from a racing duplicate key also compensates and returns the winning order.
5. Event	OrderEvent is published only after the commit succeeds (@TransactionalEventListener AFTER_COMMIT).

5.5 Order history and cancellation

- **GET /api/orders**: newest first, paged, only the caller's orders. **GET /api/orders/{id}**: 404 if it belongs to someone else.
- **POST /api/orders/{id}/cancel**: allowed only from PLACED or CONFIRMED. Order status is updated first (protected by @Version), then stock is released in product-service, then ORDER_CANCELLED is published.

5.6 Admin flows

API	Behaviour
GET /api/admin/products	All products including inactive ones
POST /api/admin/products	Create; validated (price > 0, hex colours, image URL format, sizes); 201
PUT /api/admin/products/{id}	Update; @Version prevents two admins silently overwriting each other
DELETE /api/admin/products/{id}	Soft delete (active = false): hidden from the shop, old orders stay valid; 204
GET /api/admin/orders?status=	All orders, optional status filter
PATCH /api/admin/orders/{id}/status	State machine below; illegal moves return 409. Setting CANCELLED also releases stock

```
Order state machine:
  PLACED ----> CONFIRMED ----> SHIPPED ----> DELIVERED
    |              |
    +--> CANCELLED <+          (DELIVERED and CANCELLED are final)
```

5.7 Complete API reference

Method	Path	Auth	Service	Purpose
POST	/api/auth/register	-	user	Create customer, returns JWT
POST	/api/auth/login	-	user	Returns JWT
GET	/api/users/me	user	user	Profile
GET	/api/products	-	product	Search + filter + sort + paging
GET	/api/products/{id}	-	product	Details
GET	/api/products/collections	-	product	Collection names
GET	/api/customizations	-	product	Options grouped by part
GET/POST/PUT/DELETE	/api/admin/products[/{id}]	admin	product	Product CRUD (soft delete)
POST	/internal/inventory/quote | reserve | release	internal	product	Called only by order-service
GET	/api/cart	user	order	Cart with live prices
POST	/api/cart/items	user	order	Add (merges identical lines)
PATCH / DELETE	/api/cart/items/{id}	user	order	Change quantity / remove
DELETE	/api/cart	user	order	Empty cart
POST	/api/orders/checkout	user	order	Place order (Idempotency-Key header)
GET	/api/orders, /api/orders/{id}	user	order	History, details
POST	/api/orders/{id}/cancel	user	order	Cancel, stock returned
GET	/api/admin/orders	admin	order	All orders
PATCH	/api/admin/orders/{id}/status	admin	order	Advance status

Status codes used consistently: 200 OK, 201 Created, 204 No Content, 400 validation / bad input, 401 not logged in, 403 not allowed, 404 not found, 409 business rule (duplicate email, insufficient stock, illegal status change), 503 product-service unavailable, 500 unexpected.

6. Database structure

Database per service: one PostgreSQL 16 instance holds three databases, and each service can only reach its own. Services refer to each other's data by ID only (no cross-database foreign keys, no shared tables).

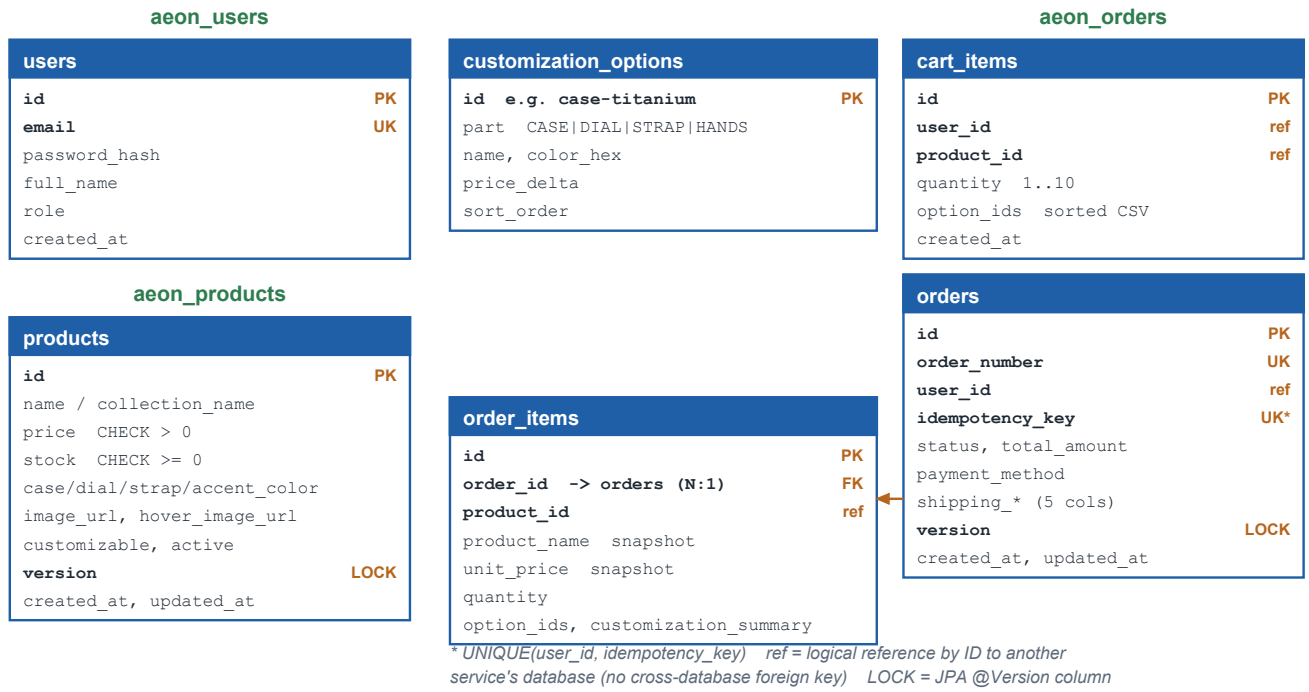


Figure 3. Tables per database. PK primary key, UK unique, FK foreign key.

6.1 Tables in detail

aeon_users.users

Column	Type	Notes
id	BIGINT identity	Primary key
email	VARCHAR(150)	UNIQUE, stored lower-case
password_hash	VARCHAR(100)	BCrypt hash
full_name	VARCHAR(100)	
role	VARCHAR(20)	CUSTOMER or ADMIN
created_at	TIMESTAMPTZ	Set by @PrePersist

aeon_products.products

Column	Type	Notes
id	BIGINT identity	Primary key
name, collection_name	VARCHAR(120), VARCHAR(60)	Indexed: collection_name
description	VARCHAR(2000)	
price	NUMERIC(12,2)	CHECK (price > 0); INR; indexed with active
stock	INTEGER	CHECK (stock >= 0): last line of defence against overselling
case_material, movement, diameter_mm, water_resistance_m	VARCHAR / INTEGER	Specifications
case_color, dial_color, strap_color, accent_color	VARCHAR(7)	Hex colours
image_url, hover_image_url	VARCHAR(500)	Added in V3
customizable, active	BOOLEAN	active = false means soft-deleted
version	BIGINT	JPA @Version (optimistic lock); also bumped by stock updates
created_at, updated_at	TIMESTAMPTZ	@PrePersist / @PreUpdate

aeon_products.customization_options

Column	Type	Notes
id	VARCHAR(40)	Readable key, e.g. case-titanium
part	VARCHAR(10)	CASE, DIAL, STRAP or HANDS (Java enum)
name, color_hex	VARCHAR	Display name and swatch colour
price_delta	NUMERIC(12,2)	Added to the base price
sort_order	INTEGER	Display order

aeon_orders.cart_items, orders, order_items

Table	Key columns and constraints
cart_items	id PK; user_id (indexed); product_id; quantity CHECK 1..10; option_ids (sorted CSV via a JPA AttributeConverter); created_at
orders	id PK; order_number UNIQUE (WD-20261009-A1B2C3); user_id; idempotency_key with UNIQUE(user_id, idempotency_key); status; total_amount NUMERIC(12,2); payment_method; shipping_name/address/city/postal_code/country; version (@Version); created_at, updated_at. Indexes: (user_id, created_at DESC), (status)
order_items	id PK; order_id FK -> orders ON DELETE CASCADE (indexed); product_id; product_name and unit_price copied at purchase time (snapshot); quantity; option_ids; customization_summary

6.2 Migrations (Flyway)

Service	Script	What it does
user-service	V1__create_users.sql	users table
product-service	V1__create_catalog.sql	products, customization_options, indexes, CHECK constraints
product-service	V2__seed_catalog.sql	10 watches in 4 collections, 16 customization options
product-service	V3__product_images_and_inr_prices.sql	image columns, photos, INR prices
order-service	V1__create_orders.sql	cart_items, orders, order_items, unique and FK constraints, indexes

Each database has a flyway_schema_history table recording applied scripts. Hibernate runs with `ddl-auto: validate`, so the application refuses to start if the entities and the schema do not match. The admin user is

seeded at startup by user-service (idempotent).

7. Concurrency, consistency and events

Risk	How we handle it	Where
Overselling when many people buy the last units	Single atomic SQL: UPDATE products SET stock = stock - :q WHERE id = :id AND active AND stock >= :q. 0 rows updated = not enough stock. No read-then-write gap.	ProductRepository.decrementStock
Stock below zero	CHECK (stock >= 0) constraint as a final guard	V1__create_catalog.sql
Deadlock between two multi-item checkouts	Lines reserved in ascending productId order, so locks are always taken in the same order	InventoryService.reserve
Partial reservation	Whole reservation in one @Transactional method; any failure rolls back all lines	InventoryService.reserve
Two admins editing one product	JPA @Version optimistic locking	Product entity
Admin status change racing a customer cancel	@Version on Order; loser gets 409 "modified concurrently, please retry"	OrderService.transition
Double click / retry creates two orders	Idempotency-Key header + UNIQUE(user_id, idempotency_key)	OrderService.checkout
Order saved but stock not reserved (or vice versa)	Saga: reserve first, save second, release (compensate) if saving fails	OrderService.checkout
Event sent for an order that rolled back	@TransactionalEventListener(phase = AFTER_COMMIT)	OrderEventListener

Proof in tests: ProductApiIT.concurrentReservationsNeverOversell starts **20 parallel reservations against a stock of 3**; exactly 3 succeed and the final stock is 0. ProductApiIT.reserveIsAllOrNothing shows a failing second line leaves the first line's stock unchanged.

> **Say:** *"We do not read the stock and then write it back, because two requests could read the same number. The decrement and the check happen in one SQL statement, so the database guarantees correctness."*

7.1 Events and messaging

OrderEvent {type, orderId, orderNumber, userId, status, totalAmount, occurredAt} with types ORDER_PLACED, ORDER_STATUS_CHANGED and ORDER_CANCELLED, published through Spring's ApplicationEventPublisher after commit. Today the listener writes a structured log line. This listener is the planned place for a message broker: roadmap phase 2 adds a transactional outbox table and Kafka, so a notification service can consume the events.

7.2 Scalability

- Services are **stateless**: identity is in the JWT, cart and orders are in the database. Run more instances and they register with Eureka automatically.
- The gateway and order-service use **client-side load balancing** (Spring Cloud LoadBalancer) across instances.
- Database per service means each database can be scaled or tuned independently.
- Timeouts on service-to-service calls (2 s connect, 5 s read) stop a slow product-service from hanging order-service; failures become 503.

7.3 Error handling, validation and logging

```
{
  "timestamp": "2026-10-09T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/auth/register",
  "correlationId": "5f0c2b1e-...",
  "fieldErrors": { "email": "must be a well-formed email address" }
}
```

- **One error shape** from every service and from the gateway (ApiError record + GlobalExceptionHandler in common).
- **Validation**: Bean Validation annotations on request records, triggered by @Valid. Nested errors are reported like shipping.city.
- **Logging**: the gateway creates an X-Correlation-Id; every service puts it in the log MDC, so all log lines for one request can be found across services. One access log line per request (method, path, status, time).
- **Business logs**: registrations, logins, cart changes, reservations, status changes, compensation failures. Passwords and tokens are never logged.

8. Testing, Docker and CI/CD

8.1 Test pyramid

Level	Tools	What is covered	Count
Unit (backend)	JUnit 5, Mockito, AssertJ	Pricing rules, cart merge, saga compensation, idempotent replay, status machine, JWT, gateway route policy	45
Integration (backend)	Spring Boot Test, MockMvc, Flyway on H2	Real HTTP + DB per service: validation, status codes, search/sort/paging, admin roles, concurrency, full purchase journey	21
Frontend	Vitest, Testing Library	API client, formatting, components, validation, products page filters	18
Functional / E2E	Postman + Newman	Real stack through the gateway: happy paths, negative paths, idempotency, header spoofing, admin flows	43 req / 116 assertions

Naming: classes ending in **Test** run in Maven's test phase (Surefire); classes ending in **IT** run in the integration-test phase (Failsafe). JaCoCo writes coverage during verify.

8.2 Docker

- **backend/Dockerfile**: one multi-stage file for every module (build arg SERVICE). POM files are copied first so dependencies are cached; the runtime image is a small JRE running as a non-root user.
- **frontend/Dockerfile**: node builds the app, nginx serves it and proxies /api.
- **docker-compose.yml**: postgres -> discovery -> user/product -> order -> gateway -> frontend, each waiting for the previous one's health check. db/init.sql creates the three databases. Only frontend, gateway, Eureka and Postgres expose ports.

8.3 CI/CD pipeline (GitHub Actions)

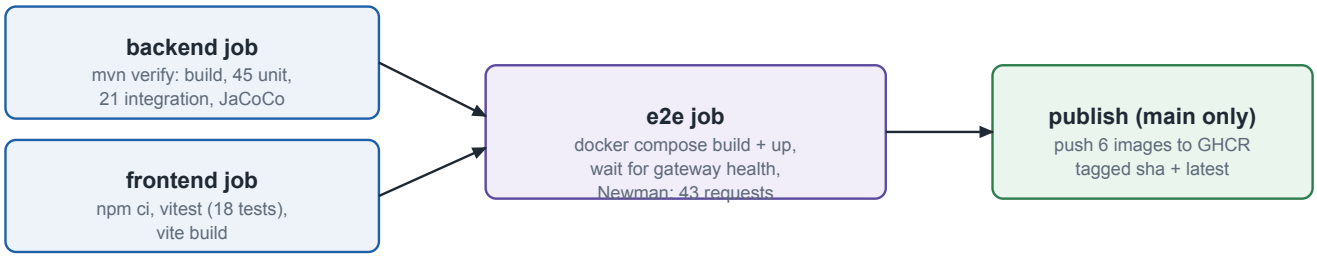


Figure 4. .github/workflows/ci.yml runs on every push and pull request.

- Test reports are published as GitHub check runs (JUnit XML) and uploaded as artifacts with JaCoCo coverage.
- Superseded runs on the same branch are cancelled; permissions are declared least-privilege.
- Planned (roadmap): deploy to AWS ECS Fargate + RDS PostgreSQL using GitHub OIDC, staging then production with approval.

9. Demo script (about 12 minutes)

Before the demo: run `docker compose up --build` (or the local JARs), open the web app, Swagger and Eureka in tabs, and have Postman open with the collection and environment imported.

#	Show	What to say
1	Eureka dashboard (:8761)	Four applications registered: gateway and three services. They find each other by name.
2	Products page	Public browsing. Search "diver", filter by collection and price, sort by price, paging. All from one search API built from JPA Specifications.
3	Register, then login	BCrypt password, JWT returned. Try a duplicate email to show the 409 message.
4	Customize page	Pick a base watch and options. The total is shown, but the server recalculates the price; the browser only sends option IDs.
5	Cart	Add the same build twice: it merges into one line. Change quantity; prices are refreshed from product-service.
6	Checkout	Submit with empty fields to show server validation messages. Then place the order: stock reserved atomically, order saved, cart cleared.
7	My Orders	Order number WD-..., status PLACED. Cancel it: stock goes back to product-service.
8	Admin (log in as admin)	Create a product with an invalid colour to show field errors, then create it properly; edit; delete (soft delete). Move another order PLACED -> CONFIRMED -> SHIPPED; try an illegal jump to show 409.
9	Swagger UI (:8080/swagger-ui.html)	One page for all three services (gateway aggregates them). Authorize with a token and call /api/cart.
10	Postman / Newman	Run the collection: 43 requests, 116 assertions pass, including the spoofed header test.
11	Logs	Pick a correlation ID from a response header and find the same ID in gateway and service logs.
12	Tests and pipeline	mvn verify output (45 unit + 21 integration), especially the 20-thread no-oversell test; walk through ci.yml.

Backup plan if something fails live

- If Docker is slow: run the JARs directly (README section 4, Option B).
- If an image or video CDN is blocked on the office network, the page shows a placeholder image; the functionality is unaffected.
- Keep docs/TEST_RESULTS.md open as evidence of passing runs.

10. Likely panel questions and answers

Short, direct answers. Where useful, mention the class name so the panel can see it in the code.

Architecture

Q. Why microservices and not a monolith?

The brief asked for microservices, and the domain splits cleanly: identity, catalog with stock, and orders. Each part can be deployed and scaled on its own (catalog reads are much more frequent than checkouts) and owns its data. The cost is more moving parts and distributed transactions, which we handle with the saga.

Q. What does the API gateway do? Could you skip it?

It is the single entry point: routing, JWT validation once for all services, removing spoofed headers, CORS, correlation IDs and one Swagger page. Without it the client would need to know every service address and each service would have to validate tokens itself.

Q. What is the "common service"? Why a library instead of a running service?

common is a shared JAR used by every service: the error format and global exception handler, JWT utility, identity-header constants and checks, correlation-ID filter and pagination envelope. These are cross-cutting rules that must behave the same everywhere. Making them a network service would add a network call to every request for no benefit. Spring Boot auto-configuration wires it in automatically.

Q. How does service discovery work? What if an instance dies?

Each service registers with Eureka on start and sends heartbeats. The gateway uses lb://service-name and Spring Cloud LoadBalancer picks a live instance from the registry. A dead instance stops sending heartbeats and is removed; with several instances traffic moves to the healthy ones.

Q. How is MVC applied?

Model = JPA entities, View = JSON DTO records, Controller = @RestController classes. Business logic and transactions are in the service layer, data access in repositories. Controllers never touch repositories and entities never leave the service.

Security

Q. How does authentication work end to end?

Login checks the BCrypt hash and returns an HS256 JWT (user id, email, role, 24 h). The browser sends it as a Bearer header. The gateway verifies it and forwards the identity as X-User-* headers to the service.

Q. Why validate the JWT only at the gateway?

One place for auth logic and one shared secret to protect. Services are not reachable from outside (only the gateway publishes a port), and they still check roles themselves as a second layer. In production we would add mTLS or network policies.

Q. Can a user fake the X-User-Id or X-User-Role header?

No. The gateway deletes any client-supplied X-User-* headers before adding the verified ones. There is a Postman test: a customer token with X-User-Role: ADMIN still gets 403.

Q. How are passwords stored?

BCrypt hashes (salted, slow by design) using spring-security-crypto. Password rules: 8-72 characters with a letter and a digit. Login gives the same error for wrong email and wrong password.

Q. Where is the token stored in the browser? Is that safe?

localStorage, for simplicity. The risk is XSS, which React reduces by escaping output. The planned improvement is short-lived access tokens with an httpOnly refresh cookie (roadmap phase 4).

Data and transactions

Q. Why a database per service? How do you get data from two services?

Each service owns its schema, so one team or service can change it without breaking another. We never join across databases; order-service asks product-service over REST (quote, reserve). Orders copy the product name and price they need as a snapshot.

Q. How do you handle a transaction across order-service and product-service?

A saga with compensation: reserve stock (one transaction in product-service), save the order (one transaction in order-service), and if saving fails, call release to give the stock back. We avoid two-phase commit because it locks resources across services and is poorly supported by modern infrastructure.

Q. What happens if the compensation call itself fails?

It is logged with a searchable COMPENSATION FAILED marker for reconciliation. The roadmap adds a transactional outbox so the release is retried automatically until it succeeds.

Q. Why store price and name in order_items?

An order is a record of what was sold and for how much. If an admin later changes a price or renames a product, past orders must not change.

Q. Why soft delete products?

Orders and carts reference product IDs. Setting active = false hides the product from the shop but keeps history valid.

Q. What is Flyway and why ddl-auto: validate?

Flyway runs numbered SQL scripts once per database and records them in flyway_schema_history, so every environment has the same schema. Hibernate only validates the schema; it never changes it, which avoids accidental schema changes in production.

Concurrency

Q. How do you stop two customers buying the last watch?

The stock decrement is one conditional UPDATE: stock = stock - q WHERE stock >= q. The database executes it atomically, so only one can succeed. 0 rows updated means insufficient stock. A test fires 20 parallel reservations at a stock of 3 and exactly 3 succeed.

Q. Why not SELECT ... FOR UPDATE, or optimistic locking with retries, for stock?

Both work. The conditional UPDATE needs one round trip, holds the row lock only for that statement, and needs no retry loop. We do use @Version optimistic locking where a whole entity is edited (admin product edits, order status changes).

Q. What is an idempotency key and why do you need it?

A unique ID the client sends with checkout. If the same request arrives twice (double click, network retry), the server returns the original order instead of creating a second one. It is enforced by a UNIQUE(user_id, idempotency_key) constraint, so even two simultaneous duplicates cannot both succeed.

Q. How do you avoid deadlocks when two carts contain the same products?

Reservation always processes lines in ascending product ID order, so two transactions lock rows in the same order and cannot wait on each other in a cycle.

Messaging, scaling and resilience

Q. The use case mentions events and messaging. Where are they?

Order lifecycle changes raise an OrderEvent, published only after the transaction commits. Today the listener logs it. The next step is a transactional outbox plus Kafka, so other services (notifications, analytics) can react without coupling to order-service.

Q. How would the system scale?

All services are stateless, so we add instances; they register with Eureka and are load-balanced automatically. Catalog reads could also be cached (Redis) and the databases scaled separately.

Q. What if product-service is down during checkout?

The client call has timeouts (2 s connect, 5 s read). A connection failure or 5xx becomes 503 "Product service is temporarily unavailable" and no order is created. A circuit breaker (Resilience4j) is on the roadmap.

Quality and delivery

Q. What is the difference between your unit and integration tests?

Unit tests check one class with mocked dependencies (fast, many cases). Integration tests start the whole Spring context with the real Flyway schema and call the HTTP endpoints through MockMvc. Postman/Newman then tests the running system from outside.

Q. Why H2 in integration tests and not PostgreSQL?

Speed and no Docker requirement on developer machines. H2 runs in PostgreSQL mode with the same Flyway scripts. The CI e2e job tests against real PostgreSQL. Testcontainers is planned.

Q. What does the pipeline do?

On every push: backend build with unit, integration tests and coverage; frontend tests and build; then all Docker images are built, the stack is started with Compose, and the Postman collection runs against it. On main the images are pushed to GHCR.

Q. How would you deploy to AWS?

Images to ECR, services on ECS Fargate behind an Application Load Balancer, RDS PostgreSQL, secrets in Secrets Manager, GitHub Actions deploying through OIDC with staging and production stages.

Q. How do you trace a problem across services?

Every response has an X-Correlation-Id; the same ID is in every log line of every service that handled the request, and in the error body. Search the logs for that ID.

Q. How is the API documented?

springdoc generates OpenAPI from the controllers and annotations. Each service exposes /v3/api-docs; the gateway routes those and shows all three in one Swagger UI with a service selector.

11. Assumptions, limitations and next steps

- Payments are simulated (CARD / UPI / COD); no card data is collected.
- Stock reservation is synchronous REST; Kafka with an outbox is the next phase.
- Services trust gateway headers inside a private network; production adds mTLS or network policies.
- JWT is kept in localStorage; httpOnly refresh cookies are planned.
- Prices are INR inclusive of taxes; tax breakdown and shipping charges are not modelled.
- Product photos are Unsplash stock images used as placeholders for the fictional catalogue.
- The web UI is intentionally simple; the focus of the project is the backend.

Roadmap summary

Phase	Theme
1 (done)	Microservices, simple UI, tests, Docker, CI
2	Kafka + transactional outbox, reservation expiry, notification service, Resilience4j, Redis cache, Testcontainers
3	AWS deployment: Terraform, ECS Fargate, RDS, CloudFront, OIDC deploys, observability (OpenTelemetry)
4	Real payments and refunds, refresh tokens, rate limiting, security scanning
5	Rich storefront design, recommendations, admin analytics

Where to find things in the repository

Topic	Path
Setup and overview	README.md
Full technical document	docs/TECHNICAL_DOC.md
Test evidence	docs/TEST_RESULTS.md
Roadmap	ROADMAP.md
Postman collection + environment	postman/WatchesDen.postman_collection.json, postman/WatchesDen.local.postman_environment.json
Pipeline	.github/workflows/ci.yml
Gateway security	backend/api-gateway/src/main/java/com/aeon/gateway/security/
Checkout saga	backend/order-service/src/main/java/com/aeon/order/service/OrderService.java
Atomic stock update	backend/product-service/src/main/java/com/aeon/product/repository/ProductRepository.java
Database migrations	backend/*/src/main/resources/db/migration/
Concurrency test	backend/product-service/src/test/java/com/aeon/product/ProductApiIT.java